Task 4: Login Interface, JWT-Protected Account Area, and Web Security Basics

Phase 2, Task 4 deliverable. Author: Ahmed. Date: 2026-11-12. Branch: `feature/auth-account-security`.

Try it against the official API:

```
python3 api/techsphere/server.py 4010                 # the TechSphere API from the kit
VITE_API_BASE=http://127.0.0.1:4010 npm run start     # the site on http://127.0.0.1:8080
```

Open **Sign in** in the menu and use `student@techsphere.test` / `Passw0rd!`. Without `VITE_API_BASE` the site uses our own mock on `/api` (same account). The security note is **`docs/SECURITY.md`**.

1. What Was Built

Requirement	Where	Proof
Login and register forms, client-side validation, accessible errors	`login.html`, `register.html`, `components/auth-form.js`, `form-errors.js`, `utils/validate.js`	e2e "forms are labelled and validated before anything is sent"; unit `validateLogin`/`validateRegister`
POST `/auth/login` and `/auth/register`, 400 and 401 handled separately	`api/auth.js`, `authErrorMessage` in `components/auth-form.js`	e2e 401 and 400/201 journeys against TechSphere
Token stored by a documented choice with the XSS trade-off	`sessionStorage` via `createSessionStore` (`state/stores.js`); `docs/SECURITY.md` §1	unit session store; e2e sign-out
`Authorization: Bearer` on GET `/account/orders` and the address endpoints	`api/account.js` through `api/http.js` (`getToken`)	e2e checks the TechSphere request log: every `/account/*` call has `auth: true`
Token expiry: redirect to login with a clear message	`components/auth-session.js` (`watchExpiry`, `sendToLogin`), `utils/jwt.js`	e2e expired-session and 401 journeys
Logout clears token and account state, keeps the cart	`#sign-out` in `components/account-view.js`; `session.signOut()`	e2e "sign-out … keeps the cart and saved items"
Credentials never in logs or URLs	POST bodies, header only, ESLint `no-console`	e2e leak journey (browser + server log)
[v1.1] Security note: XSS, CORS, CSRF, proposed CSP	`docs/SECURITY.md` §2–§5; CSP in `netlify.toml`	`qa/headers.js`: 0 CSP violations on every page

Account page (`account.html`): greeting, order history (5 per page, Previous/Next, dates in one format although the API sends four, totals in EGP, "Cancel order" while processing), address book (edit city/street inline, delete), sign out. Private pages are `noindex`, out of the sitemap and disallowed in `robots.txt`. The menu on every page shows **Sign in** or **Account**.

2. Requests and Responses

Call	Sent	Success	Failure → what the user sees
`POST /auth/login`	JSON body `{email, password}`	200 `{token, user}` → account (or the page that sent you)	**401** → "The email or password is incorrect. Check them and try again." in the summary; the same message for both, so it does not reveal which one was wrong. A 400 (if an API sends one) shows the server's field message
`POST /auth/register`	`{name, email, password}`	201 `{token, user}` → signed in, account	**400 "Email already registered"** → on the email field: "This email is already registered. Sign in instead."; other 400s (bad email, short password) → the server message
`GET /account/orders?offset&limit=5`	`Authorization: Bearer …`	list + total → pages	401 → expiry flow (§3); 500/network → retry message
`PATCH /account/orders/{id}/cancel`	Bearer	200 → status "Cancelled"	**409** → "This order can no longer be cancelled."; 404 → "That item no longer exists. The list has been refreshed."
`GET /account/addresses`	Bearer	list	401 → expiry flow

Call	Sent	Success	Failure → what the user sees
`PUT /account/addresses/{id}`	Bearer, `{city, street}`	200 → "Address saved: …"	empty city stopped in the form ("Enter the city."); 400/404 → message
`DELETE /account/addresses/{id}`	Bearer	204 → removed, "Address deleted"	message, address kept

Client-side validation runs first, so an empty or malformed form is never sent: email format, password required (login); name, email, password ≥ 8 characters, matching confirmation (register). Errors are shown next to each field (`aria-invalid`, `aria-describedby`) and in a summary with `role="alert"` that receives focus and links to each field.

3. Token Flow and Expiry

1. Sign-in succeeds → `{token, expiresAt (from the JWT exp claim), user}` saved in `sessionStorage` (`brandstore:session:v1`). The password is never stored.
2. Every account call adds `Authorization: Bearer <token>` in `api/http.js`; nothing else does.
3. Before the account page loads, an expired session is removed and the shopper goes to `login.html` with **"Your session has expired. Please sign in again."** After signing in they return to the account.
4. The same happens if the server answers **401** to any account call, and a timer does it if the token expires while the page is open.
5. Opening `account.html` signed out → "Please sign in to see your account."

The notice and the return page travel in a one-visit sessionStorage flag, not in the URL; the return page must be one of our own pages (no open redirect).

4. Logout

"Sign out" removes the session key and the flags, clears the account page and goes to `login.html` with "You have signed out. Your cart and saved items are still on this device." The cart (`brandstore:cart:v1`) and saved items (localStorage) are not touched; the e2e journey checks both before and after.

5. Security Notes (summary of `docs/SECURITY.md`)

- **Storage:** `sessionStorage` — survives a refresh, not closing the tab. **Honest risk:** script injected into the page (XSS) could read and use the token until it expires. HttpOnly cookies would avoid that but need the server to set them.
- **XSS:** all text goes through `textContent`; `innerHTML`, `eval`, inline scripts and inline handlers are banned (`qa/js-rules.js`); CSP `script-src 'self'`.
- **CORS:** the TechSphere server echoes our origin and allows only `Authorization` and `Content-Type`, so the Bearer header works and the app does not send `X-Client-Id` there.
- **CSRF:** browsers attach cookies automatically, never an `Authorization` header, so a forged cross-site request arrives without our token. With cookies we would need `SameSite` plus a CSRF token.
- **CSP (deployed):** `default-src 'self'; script-src 'self'; style-src 'self'; img-src 'self'; font-src 'self'; connect-src 'self'; form-action 'self'; base-uri 'self'; frame-ancestors 'none'; object-src 'none'; upgrade-insecure-requests` — add the API origin to `connect-src` when it is on another host.

6. QA Matrix

Check	Result
`npm run test:node` (unit + integration, incl. the TechSphere server)	**177 of 177** pass
`npm run e2e` (Chromium desktop + phone)	**71 of 71** journey runs; 197 `/api` requests checked against the spec, 0 problems
— of which `auth:` journeys against the official TechSphere server	**15 of 15** (8 journeys; the account-requests one runs on desktop only)
`npm run contract`	54 requests, 9 operations, 0 problems
`npm run lint`, `format:check`	clean
`npm run build`	OK, 69 modules reachable
`qa/js-rules`, `headers` (with CSP), `layout`, `keyboard`, `behaviour`	0 failures

Check	Result
`qa/preflight`	475 passed, 0 failed
`qa/contrast`	lowest ratio 5.02, 0 below threshold
`qa/cls`	CLS 0

The `auth:` journeys:

Journey	What it proves
forms are labelled and validated	every input has a label; empty/bad input shows field errors and the summary; **no request is sent**
wrong credentials (401)	one clear message, no account data; correct credentials open the account
registration	validation, 400 "already registered" on the email field, 201 signs in
account requests carry the Bearer token	orders pages, cancel 200 and 409, address PUT and DELETE; server log shows `auth: true`
expired session	a stored token with a past `exp` → login with the expiry message → back to the account
server 401 / expiry while open	token rejected by the server, and a token expiring on the open page, both redirect
sign-out	session and account data gone; cart and saved items still there
no leaks	token and password are absent from every request/navigation URL, the console and the server's request log (shown to fail when a token is logged on purpose)

7. Deviations and Limits

- **No "add address":** the TechSphere API has no create endpoint, so the address book edits and deletes only. Our own mock has the same shape.
- **Our mock vs TechSphere:** our mock wraps bodies in `{data}` and uses cents; `src/api/techsphere.js` translates, so the same UI works on both. Grading uses `VITE_API_BASE`.
- **No 400 from TechSphere login:** it returns 401 for any wrong or missing credentials; the 400 path is shown by registration and handled for login in case another API sends it.
- **Token not verified in the browser:** the JWT is only decoded to read `exp`; the server decides validity.
- **Tests:** Vitest cannot be installed in this environment; `npm run test:node` runs the same files. The TechSphere tests and journeys need Python 3 and are skipped (with a note) without it.

Appendix: Security note (docs/SECURITY.md)

Plain-language notes on how BrandStore signs people in and what protects them. Each point names the code that does it and the check that proves it.

1. Where the token is kept, and the risk we accept

Choice: `sessionStorage` (key `brandstore:session:v1`: token, expiry time, name/email/role).

Option	Survives refresh	Survives closing the tab	Readable by injected script (XSS)
Memory only	no (signed out on every refresh)	no	yes, while the page is open
sessionStorage (chosen)	**yes**	**no**	**yes**
localStorage	yes	yes (stays until logout)	yes, and for much longer
HttpOnly cookie (needs the server)	yes	configurable	**no**

The honest trade-off: any script running in our page can read sessionStorage, so **if an attacker managed to run JavaScript on the site (XSS), they could steal the token** and use it until it expires (1 hour on the TechSphere API). We accept this because:

- a refresh should not sign the shopper out (that rules out memory only);
- the token should not outlive the tab on a shared computer (that rules out localStorage);
- the API returns the token in the response body, so an HttpOnly cookie is not possible from the browser side; it is the recommended next step on the server.

We reduce the risk instead: no way to inject script (section 2), a strict CSP (section 5), a short token life, and the session is removed on sign-out, on expiry and on any 401. The password is never stored anywhere.

Expiry. The token's `exp` claim is read (`src/utils/jwt.js`; only to know the time, a browser cannot verify a signature). An expired session is never sent: the account page sends the shopper to sign-in with "Your session has expired. Please sign in again." A token the server rejects (401) does the same, and a timer does it if the token expires while the page is open (`src/components/auth-session.js`).

Sign-out removes the session and the one-visit flags (sessionStorage) and leaves the page, so account data held in memory is gone. The cart and saved items (localStorage) are deliberately kept.

2. XSS (cross-site scripting) and how the code avoids it

XSS means text from outside (an API response, a URL, a form) ends up being run as code in our page. With our token in sessionStorage, that is the main way it could be stolen. How the code avoids it:

- **Text, never HTML.** Every element is built with `document.createElement` and `textContent` (`src/utils/dom.js`). `innerHTML`, `outerHTML`, `insertAdjacentHTML` and `document.write` are banned and checked by `qa/js-rules.js`. Product names, order data and error messages from the API are always plain text.
- **No inline scripts or event-handler attributes**, no `eval` or `new Function`; every page loads one ES module file (checked by `qa/js-rules.js`). This is what lets the CSP forbid inline script entirely.
- **URLs are built safely:** ids go through `encodeURIComponent`; query strings through `URLSearchParams`.
- **No open redirect:** after sign-in the app returns only to a known page of this site (`withFlags` in `src/state/session.js`); an outside address is ignored.
- `qa/behaviour.js` renders a product whose name is `<img src=x onerror=…>` and checks it shows as text.

3. CORS (cross-origin resource sharing) and why the mock server's headers matter

A browser only lets a page read a response from **another origin** (scheme + host + port) if that server says so with CORS headers. Our site and the API are on different origins when `VITE_API_BASE` points to the TechSphere API (e.g. site on `:8080`, API on `:4010`), so CORS decides whether the app works at all.

- The TechSphere server answers with `Access-Control-Allow-Origin: <your origin>`, `Access-Control-Allow-Headers: Authorization, Content-Type` and the allowed methods, and answers the browser's preflight `OPTIONS` request with 204.

- Because `Authorization` is in the allowed headers, the browser lets us send the Bearer token. A header that is not in the list makes the browser block the request before it is sent; that is why the app does **not** send its `X-Client-Id` header to the TechSphere API (`src/api/product-api.js`).
- CORS protects **users**, not the server: it stops other websites from reading responses in a visitor's browser. It is not access control (curl ignores it). `Allow-Origin: *` is fine for public catalog data; a real API that used cookies would have to name its exact origins and never combine `*` with credentials.
- Our own mock serves the API on the **same origin** (`/api` with `npm run start`, Vite and the Netlify function), so no CORS is needed in production; its CORS headers only matter when it runs on its own port.

4. CSRF (cross-site request forgery): Bearer tokens versus cookies

CSRF is when another website makes the visitor's browser send a request to our API, and the browser attaches the visitor's credentials **automatically**. Browsers attach **cookies** automatically; they never attach an `Authorization` header by themselves. Our token is sent only by our own code in that header, so a forged request from another site arrives **without** it and gets 401: classic CSRF does not apply to this design.

The price is section 1: a token readable by script is exposed to XSS. If the API moved the token to an HttpOnly cookie (safer against XSS), CSRF would come back and need `SameSite=Lax` or `Strict` cookies plus a CSRF token or an Origin check on every request that changes data.

5. Proposed Content-Security-Policy for the deployed site

Already set in `netlify.toml` and tested by `qa/headers.js` (every page loads with 0 CSP violations):

```
Content-Security-Policy: default-src 'self'; script-src 'self'; style-src 'self'; img-src 'self';
  font-src 'self'; connect-src 'self'; form-action 'self'; base-uri 'self'; frame-ancestors 'none';
  object-src 'none'; upgrade-insecure-requests
```

Directive	What it does for us
`script-src 'self'`	Only our own script files run: no inline script, no injected `<script src=…>` from elsewhere. The strongest limit on XSS.
`connect-src 'self'`	`fetch`/Axios may only call our own origin (the `/api` function). **When deployed against an API on another origin, add it here**, e.g. `connect-src 'self' https://api.techsphere.example`; that also stops stolen data being sent to an attacker's server.
`style-src`, `img-src`, `font-src 'self'`	Only our own files.
`form-action 'self'`	Forms can only submit to our own pages.
`base-uri 'self'`, `object-src 'none'`	Block two classic injection tricks (`<base>` and plugins).
`frame-ancestors 'none'`	Nobody can frame the site (clickjacking on the sign-in form).
`upgrade-insecure-requests`	Any `http://` request is upgraded to `https://`.

Other headers already sent: `X-Content-Type-Options: nosniff`, `X-Frame-Options: DENY`, `Referrer-Policy: strict-origin-when-cross-origin` (other sites never see our paths), `Permissions-Policy`.

6. Credentials never in URLs or logs

- Forms use `method="post"` and the script stops the browser's own submission; email and password travel only in the JSON **body** of `POST /auth/login` / `/auth/register` (`src/api/auth.js`).
- The token travels only in the `Authorization` **header** (`src/api/http.js`, `src/api/account.js`).
- Notices and the return page travel in sessionStorage flags, not in the URL.
- ESLint forbids `console.log`/`info`/`warn` in `src/` (only `console.error` for a feature that fails to start).

- Checked: the e2e journey "the token and password never appear in a URL, a query string or the console" records every request and navigation URL and every console line in the browser, plus the TechSphere server's own request log, and fails if either secret appears (verified to fail when a `console.info` of the token is added).
- Fields use `autocomplete="username"`, `current-password` and `new-password` so password managers work and browsers do not save the password as ordinary form history.